

三维卷积·CUDA

三维卷积其实就是矩阵乘法多一个维度，而且还更好算，因为他是点积的和，不像矩阵还需要转一个方向所以我的思路和矩阵乘法一样，一个线程计算output中的一个元素。

在kernel中，要做的第一步就是算出当前线程负责的那个元素坐标，所以用

```
int x = blockDim.x * blockIdx.x + threadIdx.x;
int y = blockDim.y * blockIdx.y + threadIdx.y;
int z = blockDim.z * blockIdx.z + threadIdx.z;
```

来确定坐标，接下来判断越界情况，我们还需要计算出output的depth,rows,cols，然后判断，也就是

```
int output_depth = input_depth - kernel_depth + 1;
int output_rows = input_rows - kernel_rows + 1;
int output_cols = input_cols - kernel_cols + 1;

if(x >= output_depth || y >= output_rows || z >= output_cols) {
    return;
}
```

接下来就是乘法了，用三层循环计算这个元素，遍历input的(x, y, z) 到 (x + kernel_depth - 1, y + kernel_rows - 1, z + kernel_cols)，遍历kernel的(0, 0, 0) 到 (kernel_depth - 1, kernel_rows - 1, kernel_cols - 1)即可，然后每个元素相互乘起来

所以过程就是

```
float sum = 0.0f;
for(int i = 0; i < kernel_depth;i++) {
    for(int j = 0; j < kernel_rows;j++) {
        for(int k = 0; k < kernel_cols;k++) {
            sum += input[(i + x) * input_cols * input_rows + (j + y) *
input_cols + (k + z)] * kernel[i * kernel_cols * kernel_rows + j * kernel_cols +
k];
        }
    }
}
```

最后把结果给output相应的那个元素即可

```
output[x * output_rows * output_cols + y * output_cols + z] = sum;
```

完整代码

```

#include <cuda_runtime.h>

__global__ void Conv(const float *input, const float *kernel, float *output, int
input_depth,
                    int input_rows, int input_cols, int kernel_depth, int
kernel_rows,
                    int kernel_cols) {

    int x = blockDim.x * blockIdx.x + threadIdx.x;
    int y = blockDim.y * blockIdx.y + threadIdx.y;
    int z = blockDim.z * blockIdx.z + threadIdx.z;

    int output_depth = input_depth - kernel_depth + 1;
    int output_rows = input_rows - kernel_rows + 1;
    int output_cols = input_cols - kernel_cols + 1;

    if(x >= output_depth || y >= output_rows || z >= output_cols) {
        return;
    }

    float sum = 0.0f;
    for(int i = 0; i < kernel_depth; i++) {
        for(int j = 0; j < kernel_rows; j++) {
            for(int k = 0; k < kernel_cols; k++) {
                sum += input[(i + x) * input_cols * input_rows + (j + y) *
input_cols + (k + z)] * kernel[i * kernel_cols * kernel_rows + j * kernel_cols +
k];
            }
        }
    }

    output[x * output_rows * output_cols + y * output_cols + z] = sum;
}

extern "C" void solve(const float* input, const float* kernel, float* output, int
input_depth,
                    int input_rows, int input_cols, int kernel_depth, int
kernel_rows,
                    int kernel_cols) {
    // 把这里改成你的实现

    // 思路和矩阵乘法应该差不多
    // 矩阵乘法我是计算矩阵的i行j列的元素
    // 那么这里可以计算input的(x, y, z)处的元素
    // 这个元素应该需要kernel乘以
    // input的(x, y, z) 到 (x + kernel_depth - 1, y + kernel_rows - 1, z +
kernel_cols - 1)
    // 用一个三层的嵌套循环就行
    int output_depth = input_depth - kernel_depth + 1;
    int output_rows = input_rows - kernel_rows + 1;
    int output_cols = input_cols - kernel_cols + 1;

```

```
dim3 block(8, 8, 16);
dim3 grid((output_depth + block.x - 1) / block.x, (output_rows + block.y - 1)
/ block.y, (output_cols + block.z - 1) / block.z);
Conv<<<grid, block>>>(input, kernel, output, input_depth, input_rows,
input_cols, kernel_depth, kernel_rows, kernel_cols);
}
```

有个小插曲，block我一开始是取(16, 16, 16)但是都出问题了，后来想起来一个block可能也就1024个threads，会超出，所以只好改成现在这样了

运行结果

```
nsc@login:~$ nsc-run bash run_tests.sh
execution 4e7500fc-0cd0-46dc-9245-b15b6a320b5d
ok   example_3x3x3_k2x3x3: out=2x1x1 time_ms=0.103
ok   sum_2x2x2_k2x2x2: out=1x1x1 time_ms=0.011
ok   unit_kernel: out=2x2x2 time_ms=0.010
ok   zero_kernel: out=1x1x1 time_ms=0.008
ok   signed_1x2x2: out=2x1x1 time_ms=0.008
ok   seq_2x3x4_k1x2x3: out=2x2x2 time_ms=0.007
ok   uniform_4x4x4_k3: out=2x2x2 time_ms=0.009
ok   uniform_10x10x10_k3x4x5: out=8x7x6 time_ms=0.011
ok   perf_256x128x128_k5: out=252x124x124 time_ms=4.455
summary cases=9 failures=0 worst_case_ms=4.455
cuda-3d-convolution_RESULT: PASS
WARNING: Overriding HOME environment variable with APPTAINERENV_HOME is not
permitted
execution 4e7500fc-0cd0-46dc-9245-b15b6a320b5d: completed
```